

**Universidad Tecnologica de Panama
Centro Regional de chiriqui
Facultad de Sistemas Computacionales
Carrera: Lic. En Ingenieria en Sitemas y Computacion**

Asignatura:

Herramientas de Programacion IV

Tema:

Android

Profesor:

Napoleon Ibarra

Elaborado por:

Ruben Rodriguez

Cedula:

1-757-2238

Fecha de entrega:

27/3/2026

I SEMESTRE

2026

PREGUNTAS y DESARROLLO:

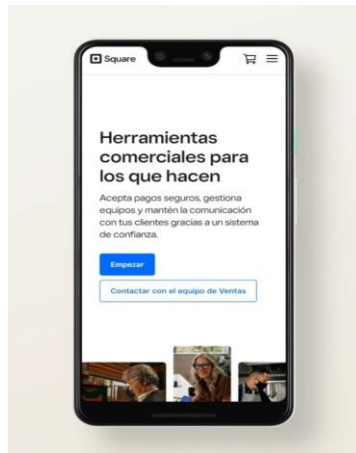
TEMA:

Glosario

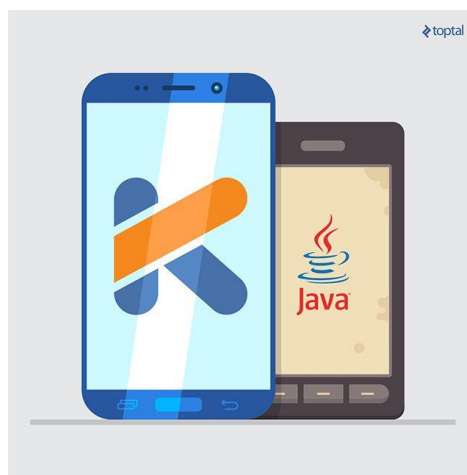
- **Apps:** Software diseñado para ejecutarse en dispositivos móviles.



- **Web para móviles:** Sitios web optimizados que se abren en el navegador del celular.



- **Nativas:** Escritas en el lenguaje específico del SO (Kotlin/Java para Android). Ofrecen el mejor rendimiento.



- **Híbridas:** Apps web "envueltas" en una carcasa nativa. Usan una sola base de código para iOS y android.



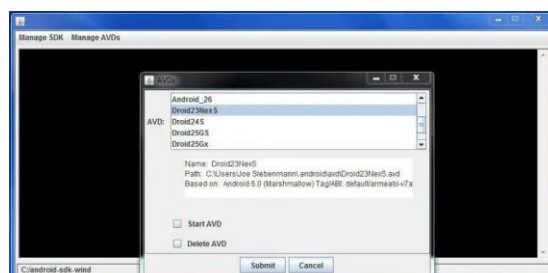
- **IDE (Android Studio):** El entorno donde escribes el código.



- **SDK (Software Development Kit):** El conjunto de herramientas y librerías necesarias para compilar la app.

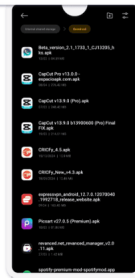


- **SDK Manager:** Herramienta dentro de Android Studio para descargar versiones específicas de Android.

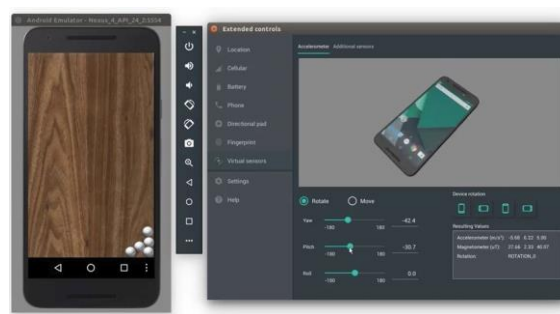


- **APK: El archivo instalable final de una aplicación Android.**

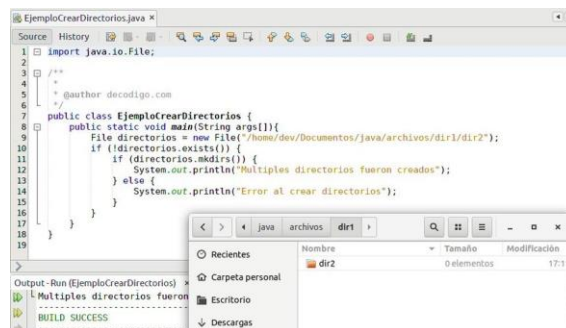
The APK file will have a .apk extension



- **Emulador: Un dispositivo virtual que corre en tu PC para probar la app.**



- **Carpeta Java: Contiene el código fuente (.kt o .java).**



- **Carpeta Res: Recursos visuales (imágenes, strings, colores).**



- **Carpeta Layout:** Archivos XML que definen cómo se ve la pantalla.



- **Android Manifest:** El "DNI" de la app; declara permisos, actividades y servicios.



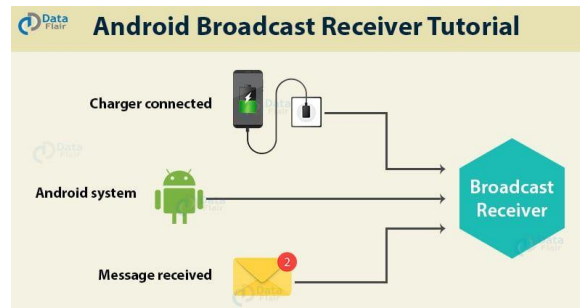
- **Activity:** Una pantalla individual de la interfaz de usuario.



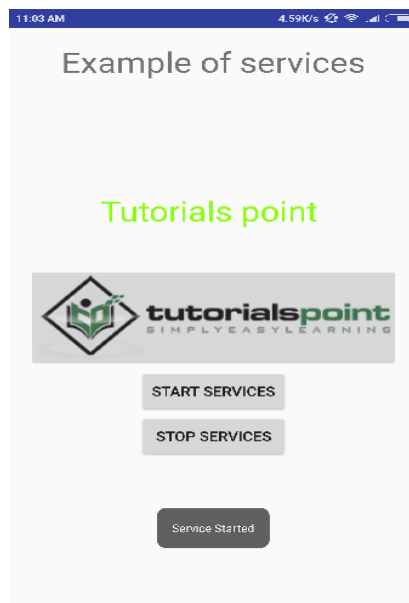
- **Fragments:** Una sección reutilizable dentro de una Activity.



- **Broadcast Receiver:** Componente que escucha eventos del sistema (ej: "batería baja").



- **Services:** Tareas que se ejecutan en segundo plano sin interfaz de usuario.



¿Cuál es la relevancia de un ciclo de vida de un programa en Android?

R// En un sistema operativo móvil, la gestión de recursos es crítica. El ciclo de vida es una máquina de estados que le indica a la aplicación qué hacer cuando el foco cambia.

Su relevancia radica en:

- **Gestión de Recursos (RAM):** Android puede "matar" procesos para liberar memoria. El ciclo de vida permite guardar datos antes de que esto ocurra para que el usuario no pierda su progreso.
- **Eficiencia Energética:** Permite pausar sensores o hilos de ejecución cuando la app no es visible (onPause / onStop), extendiendo la duración de la batería.
- **Consistencia de Estado:** Garantiza que, si el usuario rota la pantalla o recibe una notificación, la aplicación regrese exactamente al punto donde se quedó.

¿Cuál es la mejor manera de realizar una ejecución de una Aplicación Móvil?

R// Aunque el Emulador es excelente para el desarrollo inicial, la mejor manera es la ejecución en un Dispositivo Físico Real mediante depuración por USB o Wi-Fi.

¿Por qué?

- **Rendimiento Real:** Los emuladores usan la potencia de tu PC, lo que puede dar una falsa sensación de velocidad.
- **Hardware:** Solo en un dispositivo real puedes probar sensores físicos (acelerómetro, cámara real, biometría) y el consumo de batería auténtico.
- **Interacción:** La respuesta táctil (gestos, multitoque) es más precisa con los dedos que con un ratón.

¿Cuál es la estructura de un Proyecto en Android más importante? ¿Por qué?

R// La estructura más importante es el Android Manifest.

¿Por qué?

Es el archivo que actúa como el "cerebro" u "hoja de ruta" del proyecto. Sin él, el sistema operativo Android no sabría:

- **Qué componentes tiene la app:** (Qué pantallas existen).
- **Qué permisos necesita:** (Acceso a internet, cámara, contactos).

- **Cuál es la pantalla de inicio:** (La que se abre al tocar el icono).
- **Compatibilidad:** Versión mínima de Android requerida.

Qué aprendí, Qué recordé, Qué reafirmé	Resumen
¿Qué aprendí? Antes pensaba que una app era solo lo que picabas, pero aprendí que hay cosas como el Broadcast Receiver que están "orejeando" qué hace el celular (si se conecta el cargador o se queda sin red) y el Content Provider. Ya me quedó claro que las Nativas son las que sacan todo el jugo al procesador, mientras que las Híbridas son básicamente una página web disfrazada de app para ahorrar tiempo de programación. ¿Qué recordé? Recordé que Android usa este lenguaje para "dibujar" las pantallas. Es como el HTML de las páginas web, pero aplicado a los botones y menús del cel. ¿Qué reafirmé? Reafirmé que no puedes dejar procesos corriendo a lo loco. Si el usuario recibe una llamada y tu app no sabe "pausarse" con el onPause, te vas a gastar su batería y te van a desinstalar en dos segundos.	Una aplicación no es un bloque sólido, sino un rompecabezas de Activities y Layouts que viven organizados en carpetas de recursos como Res, donde guardamos desde los colores hasta los textos para no hacer un despapaye en el código de Java o Kotlin. Reafirmamos que el AndroidManifest es, sin duda, el archivo más importante de todos; es como el acta de nacimiento de la app, porque si no declaras ahí los permisos o las pantallas, el celular simplemente no sabe que existen. También quedó claro que no se trata solo de que la app se vea bonita, sino de que sea eficiente. Por eso el Ciclo de Vida es sagrado; hay que saber cuándo pausar procesos para no drenar la batería del usuario en dos segundos. Vimos que existen componentes "invisibles" como los Services o los Broadcast Receivers que hacen la chamba sucia por detrás, y que aunque el IDE (Android Studio) y el Emulador nos tiran un parote para programar, la prueba de fuego siempre será correr la app en un celular real. Solo así te das cuenta de si el touch responde chido, si el hardware aguanta o si la app de veras corre como debería en la mano del usuario.

--	--

Conclusiones

- En conclusión, lo que sacamos del glosario y la arquitectura de Android es que desarrollar una app no es solo tirar código para que se vea bonita, sino entender cómo piezas tan diferentes como las Activities, los Layouts y las carpetas de recursos se arman como un rompecabezas para que todo funcione. Nos quedó grabadísimo que el `AndroidManifest` es el que manda; si no está ahí, la app simplemente no existe para el sistema.

Además, reafirmamos que el Ciclo de Vida es sagrado para no dejar al usuario sin batería y que, aunque el IDE y el emulador nos tiran un parote en la compu, la verdad absoluta solo la tiene un dispositivo físico. Al final, el éxito de una app depende de qué tan bien estructurada esté por dentro y de que sepamos aprovechar las herramientas del SDK para que la experiencia en la mano del usuario sea fluida y no se trabe.